

vTaiwan-hono | Better Auth 登入與權限管理 工程示意圖

資訊架構 × /admin 路由守衛 × /api/* 授權四道關 × 敏感操作二次驗證 (step-up) — 依 main 分支原始碼繪製

紅實線 = Worker 端強制 (真正的安全邊界)
綠虛線 = 瀏覽器端顯示層 (UX, 非安全邊界)
黃 = 儲存與設定
△ = 已知風險或侷限

A | 資訊架構：模組分層與相依方向



△ 兩個 D1 分屬不同資料庫、無法 JOIN —— 審計日誌的操作對象姓名與原角色一律在動作生效「之前」快照 (auth-audit.ts 的 before hook)。

B | /admin 路由守衛資訊流 (SSR 請求, Worker 端安全底線)



C | /api/* 授權四道關 + 敏感操作二次驗證 (step-up) 資訊流

C-1 每個受保護端點的四道關 (順序固定, 缺一即為破口)



△ client 端的 v-if 與 auth-session.ts 只影響顯示: 任何「前端擋住就好」的做法都是漏洞。

· 除 csrf() 外, /api/query-table · /api/create-table · /api/test-al 目前無身分檢查 (bootstrap 與測試用途)。

C-2 step-up 二次驗證: 意圖寫進 OAuth state 並往返

關鍵語意: 只有「刻意為了權限而重新登入」才算通過 —— session 的 createdAt 無法承載此語意 (登出再登入也是新 session)。



D | 角色、權限與端點對照 (逐條由原始碼驗證)

D-1 角色 × 能力矩陣 (authorization.ts 的 permissionsByRole 為單一來源)

權限 / 能力	user	admin	super-admin
meeting.join	✓	✓	✓
meeting.moderate	—	✓	✓
transcription.update	—	✓	✓
成員列表 / 改角色 / 停權 (/api/auth/admin/*)	—	—	✓
變更日誌 (/api/admin/audit-log)	—	—	✓
帳號被停權 (banned) ⇒ permissions 為空, 上列全數失效			

- 不變量: 每個 Permission 都必須有實際強制它的 Worker 端點, 不留孤兒權限 (authorization.test.ts 逐字訂住權限集合)。
- 議題 (Topic) 的上架與內容修改由 talk.vtaiwan.tw (discourse) 管理員負責, 不屬本站授權範圍, 故 Permission 型別刻意不定義 topic.* 權限。
- admin 與 super-admin 的「業務權限」完全相同; 差別只在 createAuth.ts 的 adminRoleAccess (user / admin 為 userAc, super-admin 為 adminAc) 與 isSuperAdminRole()。

△ 已知死路: 二次驗證時登入了「另一個」社群帳號

若該社群帳號的 email 與管理員帳號不同, Better Auth 依 email 找不到既有使用者, 會新建一個 role='user' 的帳號並發給其 session —— 形同靜默換身分。

純函式 isWrongAccountAfterStepUp() (fresh 為真卻非管理員; 一般訪客到不了此狀態) 據此 alert 後 router.replace('/), 判定必須用伺服器回報的 fresh, 不可改用 needsStepUp (掛前碼倒數會漏判)。

侷限: provider 仍在登入狀態時, 二次驗證是零點轉跳 —— 擋得住被竊的 session cookie, 擋不住被借用的已解鎖裝置。

D-2 端點 → 授權需求 (Worker 端實際檢查的內容)

端點	授權需求
/api/me	有效 session (讀取失敗刻意 throw, 不壓成 401)
/api/auth/admin/*	401 → banned 403 → fresh 403 → 再由 plugin 判 super-admin
/api/admin/audit-log	super-admin + fresh
/api/jitsi-token (POST)	meeting.join; moderator claim 由 meeting.moderate 決定
/api/meeting/* · /api/ws/meeting/:date	meeting.join (WS 未授權者僅唯讀)
/api/transcription/:lang	meeting.join
/api/upload-transcription	transcription.update (覆蓋既有逐字稿才另需 fresh)
/api/update-outline · restore · delete	transcription.update + fresh
/api/transcriptions/:id/versions[/text]	transcription.update (舊版本不隨公開逐字稿露出)